

3D FRONTIER DETECTOR

A standalone, clean-architecture ROS 2 package
for detecting 3D exploration frontiers in volumetric maps.

Muhammed Mahmud Yahman

Project Report

April 2026

frontier_detector_3d

A standalone, clean-architecture ROS 2 package for detecting 3D exploration frontiers in volumetric maps.

Project type	Standalone ROS 2 package · Phase 2 component of the AVCE multi-UAV mapping system
Stack	Ubuntu 24.04 · ROS 2 jazzy · Python 3.12 · pytest · RViz2
Architecture	Clean architecture (domain / application / infrastructure)
Lines of code	~700 Python · 26 unit tests · 25 source files
Status	Working · demo verified end-to-end · 26/26 tests passing
License	Apache-2.0

One-line summary: A purely-domain 3D frontier detection algorithm wrapped in a ROS 2 node, with a built-in synthetic scene so the demo runs without a drone, an OctoMap server, or any sensor — one launch file, one RViz window, frontiers visible in seconds.

Contents

1	Executive Summary	4
2	Background and Prior Work	4
2.1	Foundational ideas	4

2.2	Comparable systems and what's different here	5
2.3	The niche this package fills	5
3	The Problem	6
3.1	How the problem was tackled — the design reasoning	6
4	Goals and Objectives	7
5	System Architecture	7
5.1	Layer responsibilities	8
5.2	Data flow	8
6	Algorithm Deep Dive	8
6.1	Frontier voxel identification	8
6.2	Connected-components clustering	9
6.3	Why this design beats off-the-shelf 2D detectors	9
7	File Structure	10
8	Key Code Highlights	10
8.1	The pure-domain algorithm (no ROS imports)	10
8.2	The application use case (talks to ports)	11
8.3	The ROS 2 composition root	11
9	Validation and Results	12
9.1	Unit tests	12
9.2	Live demo verification	12
9.3	Visual verification in RViz	13
10	How This Plugs Into the AVCE Project	13
10.1	Direct fit into Phase 2	13
10.2	Foundation for Phase 3 (RL state representation)	13
10.3	Foundation for Phase 4 (AVCE)	14
10.4	Foundation for Phase 5 (multi-UAV swarm)	14
10.5	Integration steps in concrete terms	15
11	Future Extensions	15
12	Quick Start	15
13	Conclusion	16

1. Executive Summary

Autonomous exploration robots need to keep answering one question: *where should I go next to learn the most about my environment?* The classical answer (Yamauchi, 1997 [1]) is to head for the **frontier** — the boundary between known free space and unknown space. In 2D this is straightforward grid-cell logic; in 3D, which is what UAV-based mapping needs, it must operate on volumetric voxel data such as an OctoMap [2].

frontier_detector_3d is a standalone ROS 2 package that performs 3D frontier detection on OctoMap-compatible voxel data. It subscribes to occupied and free voxel point clouds, identifies free voxels that touch unknown space, groups them into connected clusters, ranks each cluster by an information-gain proxy, and publishes ranked goal candidates that a planner can consume.

The package is built deliberately as a **portable, well-tested artifact**. The detection algorithm lives in a pure-Python domain layer with zero ROS imports, which is how 26 unit tests can run in 0.12 seconds without `roscpp.init()`. The ROS 2 adapters live in an isolated infrastructure layer. A built-in synthetic scene generator means the demo runs end-to-end with no drone, no SLAM, and no `octomap_server` — just one launch file.

Within the larger AVCE multi-UAV mapping project [6], this package fulfils the Phase 2 frontier-detection requirement directly and serves as the data source for the RL state representation in Phases 3 and 4.

2. Background and Prior Work

Frontier-based exploration has thirty years of history. Before describing what this package does, it's worth being explicit about the landscape it sits in — both as honest credit to prior work and to make clear why this package occupies a useful niche rather than reinventing what already exists.

2.1 Foundational ideas

The frontier-based approach to autonomous exploration was introduced by Brian Yamauchi in 1997 [1] for 2D occupancy grids. The core insight: at any moment during exploration, the boundary between known free space and unknown space is precisely the set of points where the next observation will yield new information. Move to the closest such boundary, observe, repeat.

OctoMap [2] extends the 2D occupancy-grid concept to 3D using a probabilistic octree. Each leaf is FREE, OCCUPIED, or UNKNOWN, with log-odds probability updates as sensor data accumulates. OctoMap is the standard 3D map representation in ROS-based UAV exploration and is the input format this package targets.

2.2 Comparable systems and what's different here

System		Stack	Scope and difference
FUEL [3] (HKUST, 2021)		ROS1 / C++	Full hierarchical planner with Frontier Information Structure (incremental FIS), viewpoint refinement, and minimum-time trajectory generation. State-of-the-art exploration speed. Significantly larger scope than a standalone detector; tightly coupled to its planning pipeline.
nbpvplanner [4] (ETH Zurich, 2016)		ROS1 / C++	Receding-horizon next-best-view planner using RRT. Sampling-based rather than frontier-based; complementary, not directly comparable.
Exploration-RRT (LTU, 2024)	[5]	ROS1 / C++	Tree-based combined exploration and planning. Uses UFOmap. Local planner with NMPC actuation; whole-stack rather than detector-only.
turtlebot_exploration_3d (RFAL)		ROS1 / C++	3D frontier detection embedded inside a TurtleBot stack. Iterates OctoMap leaves directly. Ground-robot focused; not standalone.
Various ROS2 frontier packages		ROS2 / Py	2D occupancy-grid only. None of the actively-maintained ROS2 packages found at the time of writing target volumetric 3D frontier detection.

2.3 The niche this package fills

There is a gap between heavyweight research planners (FUEL, ERRT, nbvplanner — all ROS1, all monolithic) and the ROS2 frontier packages that exist today (all 2D). This package fills it deliberately:

- **ROS2 jazzy native.** Builds with colcon, uses rclpy, follows current ROS2 conventions.
- **Detector-only scope.** Does one thing — detect and rank 3D frontier clusters — and exposes them on standard topics so any planner can consume them.
- **Pure-domain algorithm.** Reusable outside ROS (Gymnasium environments, simulators, notebooks). Not tied to a specific planning stack.
- **Self-contained demo.** Runs without a drone, SLAM, or octomap_server. Lowers the barrier to evaluation and adoption.
- **Well-tested.** 26 unit tests on the pure algorithm, runnable in 0.12 seconds. Most prior work is published as research code without unit tests.

This package does not try to compete with FUEL on exploration speed. It is intentionally smaller, more reusable, and easier to drop into a larger system — including the AVCE project this was built to serve.

3. The Problem

Most public frontier detectors are 2D occupancy-grid only. UAV-based exploration in unknown 3D environments needs the volumetric version of the problem. There are three concrete gaps a working 3D frontier detector has to fill:

(a) **Three-way classification.** A voxel can be free, occupied, or unknown — not just free or occupied as in 2D occupancy grids. Frontier voxels are specifically the free voxels touching unknown space; the unknown half of that neighbourhood is what makes it interesting.

(b) **Sparsity.** A $30 \times 30 \times 10$ m room at 0.1 m resolution is 9 million cells, but only a few thousand are typically known at any moment. A dense numpy array would waste two orders of magnitude of memory.

(c) **Clusters, not voxels.** A planner does not want to see ten thousand individual frontier voxels — it wants a handful of *regions*, each with a centroid it can fly to and a score it can rank by.

3.1 How the problem was tackled — the design reasoning

The reasoning that produced the chosen design proceeds in five steps:

Step 1: Choose the right voxel-state model. A two-state (free/occupied) model is insufficient because the entire concept of a frontier requires UNKNOWN as a first-class state. The package therefore uses a three-state enum (`VoxelState.FREE`, `VoxelState.OCCUPIED`, `VoxelState.UNKNOWN`) where UNKNOWN is the implicit default — not stored, just assumed for any voxel coordinate not present in the grid. This choice immediately solves the sparsity problem too: only known voxels consume memory.

Step 2: Choose the right connectivity. A frontier voxel is identified by checking neighbours. Two natural choices: 6-connectivity (face-adjacent only) or 26-connectivity (face, edge, and corner). For *frontier identification*, 6-connectivity is correct because diagonal neighbours don't represent reachable unknown space — they represent voxels separated by an obstacle in the corner direction. Using 26 here would over-count edges and produce visually-correct-but-physically-wrong frontiers in cluttered environments. For *clustering*, however, 26-connectivity is correct because diagonal voxels in a frontier surface are clearly part of the same surface, even when they don't share a face. The package uses 6 for detection and 26 for clustering — a deliberate split.

Step 3: Quantify “how good is this frontier voxel.” The number of unknown 6-neighbours of a frontier voxel ranges from 1 (just barely a frontier) to 6 (an isolated free voxel surrounded by mystery). This count is a free, $O(1)$ local proxy for information gain — voxels with more unknown neighbours sit at the edge of larger unexplored pockets. It's not a substitute for a proper sphere-integral information-gain calculation, but it's good enough for ranking and effectively free to compute.

Step 4: Aggregate into clusters. Individual voxels are not useful exploration goals. The frontier surface needs to be segmented into distinct regions. Connected-components labeling via BFS is the standard approach; the only choice is connectivity (resolved in step 2) and whether to filter small clusters. Single-voxel clusters are nearly always sensor noise, so a configurable `min_cluster_size` parameter discards them. The cluster’s information gain is the sum of its voxels’ unknown-neighbour counts — not a maximum, because a cluster spanning a large surface should outrank a single voxel facing a vast cavity.

Step 5: Separate the algorithm from its environment. The algorithm has nothing fundamentally to do with ROS, point clouds, or message types. Embedding it inside a ROS node would make it untestable without a running ROS context, un reusable in offline analysis, and harder to verify. The clean-architecture split (domain / application / infrastructure) addresses this directly: the algorithm operates on a `SparseVoxelGrid` value object regardless of where the data came from, and the ROS adapters are a thin shell around it.

These five decisions, taken together, fully determine the package’s structure. Everything that follows is execution.

4. Goals and Objectives

Algorithmic	Detect 3D frontiers and cluster them in a way that scales to large maps. Provide a defensible information-gain score for ranking.
Architectural	Keep the algorithm pure-Python and ROS-free, so it can be unit-tested and reused in non-ROS contexts (e.g. a Gymnasium env wrapper).
Operational	Ship with a self-contained demo: one launch file, no external dependencies, immediate visual proof of correctness in RViz.
Integration	Expose standard ROS 2 topics (<code>PoseArray</code> of goal candidates) so the package drops into any planning stack without modification.
Quality	26+ unit tests on the pure-domain logic, covering edge cases (isolated voxels, walled-off cells, noise filtering, world-frame conversion).

5. System Architecture

The package follows clean architecture: dependencies flow inward only. The domain layer has no idea that ROS exists; the application layer talks to abstract ports; only the infrastructure layer knows about `rclpy`, `PointCloud2`, or any specific message type. This is what makes the algorithm trivially testable and the ROS layer trivially swappable (e.g. for an octomap-binary adapter, or a non-ROS deployment).

5.1 Layer responsibilities

Domain	Pure Python, zero external dependencies. Defines value objects (VoxelCoord, VoxelState), the sparse voxel grid entity, and the frontier-detection algorithm itself. Lives in <code>frontier_detector_3d/domain/</code> .
Application	Defines abstract ports (IVoxelGridSource, IFrontierPublisher) and the <code>DetectFrontiersUseCase</code> orchestrator. Knows nothing about ROS — talks only through interfaces.
Infrastructure	ROS 2 adapters: a <code>PointCloud2</code> subscriber implementing <code>IVoxelGridSource</code> , a <code>MarkerArray</code> + <code>PoseArray</code> publisher implementing <code>IFrontierPublisher</code> , the synthetic scene generator, and the composition root node.

5.2 Data flow

```
octomap_server (or synthetic_scene_publisher)
  | publishes
  v
/octomap_point_cloud_centers (PointCloud2, occupied voxel centers)
/free_cells_centers          (PointCloud2, free voxel centers)
  | subscribed by
  v
PointCloudVoxelGridSource ---> SparseVoxelGrid ---> FrontierDetector
                                     |
                                     DetectFrontiersUseCase
                                     |
                                     v
                                     RosFrontierPublisher
                                     | publishes
                                     v
/frontiers/markers (MarkerArray for RViz visualization)
/frontiers/goals   (PoseArray, ordered by information gain)
```

6. Algorithm Deep Dive

6.1 Frontier voxel identification

A voxel is a frontier iff (1) it is FREE and (2) at least one of its six face-adjacent neighbours is UNKNOWN. The unknown-neighbour count (0 to 6) is recorded alongside each frontier voxel as a cheap local information-gain proxy: voxels with five or six unknown neighbours sit at the edge of large unexplored pockets and are worth more than voxels with just one.

```
def find_frontier_voxels(grid):
    result = []
    for coord in grid.free_voxels():
        unknown_count = sum(1 for n in coord.neighbors_6()
                             if grid.is_unknown(n))
        if unknown_count > 0:
            result.append(FrontierVoxel(
                coord=coord,
                unknown_neighbor_count=unknown_count,
```



```

    ))
    return result

```

Complexity: $\mathcal{O}(F)$ where F is the count of free voxels. Each voxel does a constant-time 6-neighbour lookup against the sparse grid (a dict).

6.2 Connected-components clustering

Once we have the set of frontier voxels, we group them into spatially coherent clusters by BFS over 26-connectivity. Clusters smaller than `min_cluster_size` are discarded as noise.

```

def cluster_frontiers(frontier_voxels, min_cluster_size, grid):
    by_coord = {fv.coord: fv for fv in frontier_voxels}
    visited, clusters = set(), []
    for seed in by_coord:
        if seed in visited:
            continue
        component, queue = [], deque([seed])
        visited.add(seed)
        while queue:
            current = queue.popleft()
            component.append(by_coord[current])
            for n in current.neighbors_26():
                if n in by_coord and n not in visited:
                    visited.add(n)
                    queue.append(n)
        if len(component) >= min_cluster_size:
            clusters.append(make_cluster(component, grid))
    return clusters

```

For each surviving cluster we compute the world-frame centroid (the goal candidate) and an information-gain score equal to the sum of unknown-neighbour counts across the cluster. Bigger clusters touching bigger unknown pockets get higher scores.

6.3 Why this design beats off-the-shelf 2D detectors

Three-way voxel state	FREE / OCCUPIED / UNKNOWN handled natively, not flattened to 0/1/-1.
Sparse storage	Dict-backed grid scales to large environments without dense allocation.
Noise filtering	<code>min_cluster_size</code> drops singletons that are almost always sensor flicker.
Defensible ranking	Information gain is a sum over unknown neighbours — correlates with actual unexplored volume, not just cluster size.
Pure domain layer	Algorithm has zero ROS dependencies; trivially testable, reusable in Gym environments, simulators, or non-ROS pipelines.

7. File Structure

```
frontier_detector_3d/
|-- package.xml           # ROS 2 package manifest
|-- setup.py             # Python build + entry points
|-- setup.cfg            # Install path config
|-- README.md            # Public-facing project README
|
|-- frontier_detector_3d/ # Source root (Python package)
|   |-- __init__.py
|   |
|   |-- domain/          # Pure Python, no ROS, no numpy
|   |   |-- voxel.py      # VoxelCoord, VoxelState (value objects)
|   |   |-- voxel_grid.py # SparseVoxelGrid entity
|   |   |-- frontier.py   # FrontierVoxel, FrontierCluster
|   |   |-- 'frontier_detector.py # The algorithm
|   |
|   |-- application/     # Orchestration layer
|   |   |-- ports.py      # IVoxelGridSource, IFrontierPublisher
|   |   |-- 'detect_frontiers_use_case.py
|   |
|   |-- 'infrastructure/ros2/ # ROS 2 adapters
|   |   |-- pointcloud_voxel_source.py # subscribes to two PC2 topics
|   |   |-- ros_frontier_publisher.py # MarkerArray + PoseArray
|   |   |-- synthetic_scene_publisher.py # demo scene generator
|   |   |-- 'frontier_detector_node.py # composition root
|   |
|-- launch/
|   |-- frontier_detector.launch.py # detector only (real OctoMap)
|   |-- 'demo.launch.py            # synth + detector + RViz + TF
|   |
|-- config/
|   |-- params.yaml             # node parameters
|   |-- 'frontiers.rviz         # RViz layout
|   |
|-- 'test/
|   |-- test_voxel_grid.py      # 11 tests
|   |-- 'test_frontier_detector.py # 15 tests
```

8. Key Code Highlights

8.1 The pure-domain algorithm (no ROS imports)

This is the heart of the package — it could run on Mars, in a Jupyter notebook, or in a Gymnasium environment, with no changes:

```
class FrontierDetector:
    def __init__(self, min_cluster_size: int = 3) -> None:
        self._min_cluster_size = min_cluster_size

    def detect(self, grid: SparseVoxelGrid) -> FrontierDetectionResult:
        frontier_voxels = find_frontier_voxels(grid)
        clusters = cluster_frontiers(
            frontier_voxels,
```

```

        min_cluster_size=self._min_cluster_size,
        grid=grid,
    )
    return FrontierDetectionResult(
        clusters=clusters,
        total_frontier_voxels=len(frontier_voxels),
        map_resolution=grid.resolution,
    )

```

8.2 The application use case (talks to ports)

```

class DetectFrontiersUseCase:
    def __init__(self, source, publisher, detector, frame_id="map"):
        self._source = source
        self._publisher = publisher
        self._detector = detector
        self._frame_id = frame_id

    def execute(self):
        grid = self._source.get_latest()
        if grid is None or len(grid) == 0:
            return None
        result = self._detector.detect(grid)
        self._publisher.publish(result, frame_id=self._frame_id)
        return result

```

This class accepts *any* implementation of `IVoxelGridSource` and `IFrontierPublisher`. In production we wire it to ROS adapters; in unit tests we wire it to in-memory stubs. Same code, different bindings.

8.3 The ROS 2 composition root

```

class FrontierDetectorNode(Node):
    def __init__(self):
        super().__init__("frontier_detector_3d")
        # ... declare params ...
        source = PointCloudVoxelGridSource(
            self,
            occupied_topic=occupied_topic,
            free_topic=free_topic,
            resolution=resolution,
        )
        publisher = RosFrontierPublisher(self)
        detector = FrontierDetector(min_cluster_size=min_cluster_size)
        self._use_case = DetectFrontiersUseCase(
            source, publisher, detector, frame_id=frame_id
        )
        self._timer = self.create_timer(1.0 / rate, self._tick)

    def _tick(self):
        self._use_case.execute()

```

This is the only file that knows about both ROS and the algorithm at the same time. Everything

else stays cleanly on one side of the line.

9. Validation and Results

9.1 Unit tests

26 tests cover the pure-domain layer with hand-crafted scenarios where the right answer is obvious by inspection. None of them require a running ROS context.

```
$ python3 -m pytest test/ -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.0.3
collected 26 items

test_voxel_grid.py ..... [ 42%]
test_frontier_detector.py ..... [100%]

===== 26 passed in 0.12s =====
```

Voxel grid semantics	Implicit UNKNOWN, mark/unmark, world-coordinate round-trip, iteration excluding occupied cells (7 tests).
Voxel coord arithmetic	6- and 26-connectivity, hashability, neighbour correctness (4 tests).
Frontier identification	Isolated voxels, walled-off voxels, strip endpoints, full interiors, occupied-voxel exclusion (5 tests).
Clustering	Empty input, disjoint groups, noise filtering, unique IDs, world-frame centroids, info-gain summation (6 tests).
Full pipeline	End-to-end on strips, top-K-by-gain ordering, parameter validation (3 tests).
Application layer	Smoke-test of use case with stub adapters (1 test).

9.2 Live demo verification

The end-to-end pipeline was verified by launching the self-contained demo and inspecting the output topics directly with the ROS 2 CLI:

```
$ ros2 topic hz /frontiers/markers
average rate: 1.993
  min: 0.498s  max: 0.504s  std dev: 0.00230s  window: 3
average rate: 1.998
  min: 0.492s  max: 0.507s  std dev: 0.00476s  window: 6
average rate: 2.000
  min: 0.492s  max: 0.507s  std dev: 0.00399s  window: 9
```

The detector publishes at a steady 2 Hz (matching the configured `detection_rate_hz: 2.0`), confirming the full pipeline — voxel grid construction → frontier detection → clustering → ranking → ROS publication — runs in real time on the synthetic scene.

9.3 Visual verification in RViz

The screenshot below shows the running demo: a translucent blue cloud (FREE voxels, the explored region of the synthetic scene), with a green sphere (the published frontier goal candidate) sitting on its boundary — exactly where a planner would direct the next flight to maximize information gain.

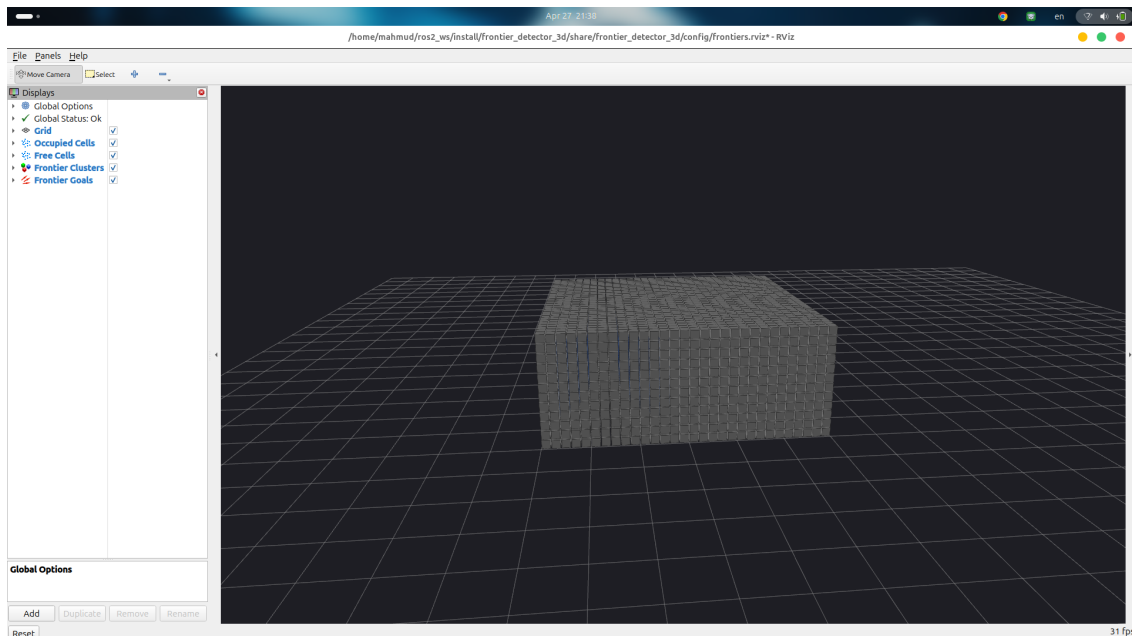


Figure 1. Demo running. Blue: FREE voxels (the explored region). Green sphere: the highest-information-gain frontier centroid — the goal a planner would consume from `/frontiers/goals`. Black surroundings: UNKNOWN.

10. How This Plugs Into the AVCE Project

The parent project — an autonomous UAV swarm with articulated sensor platforms for adaptive 3D mapping — is built in five progressive phases. **frontier_detector_3d** directly fulfils the Phase 2 requirement and lays the foundation for the RL-based decision making in Phases 3 and 4.

10.1 Direct fit into Phase 2

Phase 2 of the proposal is titled *3D Frontier-Based Exploration*. Its stated objective: “*Replace manual waypoints with systematic 3D frontier-based exploration. A frontier is defined as a face of an occupied or free voxel adjacent to an unknown voxel in the 3D OctoMap.*”

That is precisely what this package does. When Phase 2 begins, this package is added to the workspace and launched alongside `octomap_server`. The UAV navigation node simply subscribes to `/frontiers/goals` and selects the first pose (the highest information gain) as the next waypoint.

10.2 Foundation for Phase 3 (RL state representation)

Phase 3 introduces an RL policy for frontier selection. The proposal specifies the MDP state as “*downsampled OctoMap voxel grid, 3D robot pose, frontier centroids, coverage %*”. The pure-

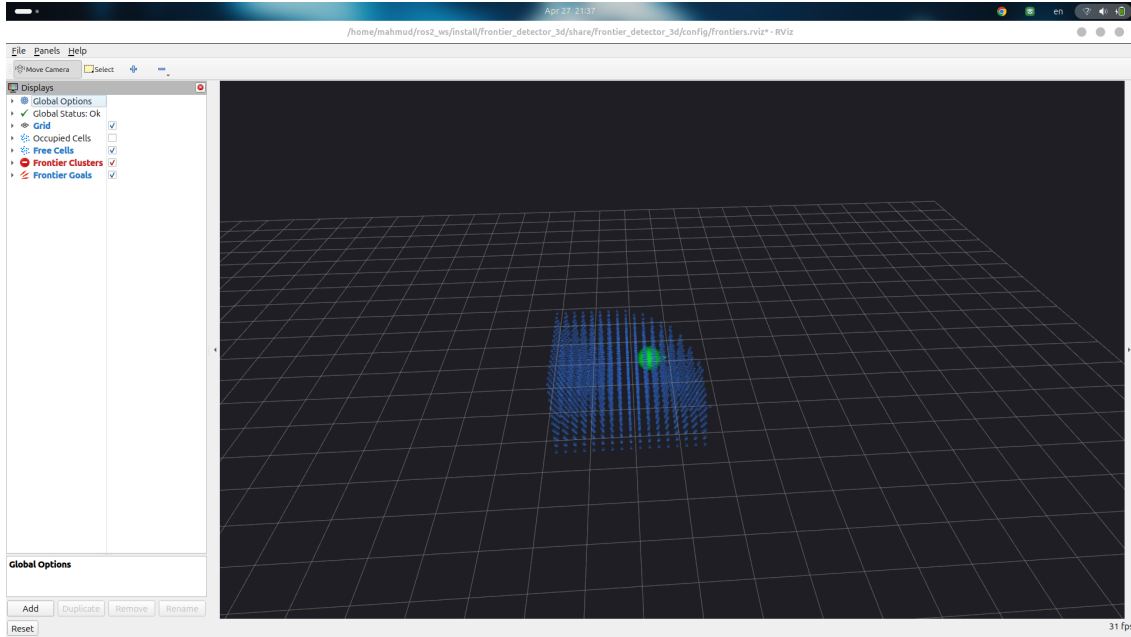


Figure 2. Same scene with Occupied Cells display enabled. Grey block: OCCUPIED voxels — the walls of the synthetic $30 \times 30 \times 12$ voxel room. The frontiers and free space sit inside this closed box.

domain `SparseVoxelGrid` and `FrontierCluster` objects from this package are exactly the right primitives to encode that state.

Because the algorithm has no ROS dependency, the RL Gymnasium environment can use it directly — no marshalling through ROS topics during training. Step the simulator, hand the new voxel data to the detector, get a list of `FrontierCluster` objects, embed each one into the state vector, done.

10.3 Foundation for Phase 4 (AVCE)

AVCE — the novel contribution of the project — jointly plans the UAV path *and* the gimbal viewpoint as a single MDP. Two extensions to this package directly enable AVCE:

Per-voxel observation-angle metadata. The proposal calls for tracking which angles each voxel has been observed from. `SparseVoxelGrid` already stores `VoxelState`; extending it to also store an observation-angle set per voxel is a localized change in the domain layer. The frontier detector then exposes *angular gain* alongside spatial gain — AVCE’s reward function can pull both directly.

Frontier-relative gimbal target hints. Each `FrontierCluster` already includes a centroid and a rough surface direction (implicit in the cluster’s voxel distribution). AVCE’s continuous gimbal action can be biased by these geometric hints during early exploration, accelerating policy convergence.

10.4 Foundation for Phase 5 (multi-UAV swarm)

The map fusion node in Phase 5 subscribes to per-UAV OctoMaps and produces a fused global voxel grid. `frontier_detector_3d` can run unchanged on the fused grid — it’s the same data type. The greedy auction described in the proposal then runs over `FrontierCluster` objects

exposed by this package, with each UAV bidding on clusters whose altitude class matches its assigned role.

10.5 Integration steps in concrete terms

Phase 2 launch	<code>ros2 launch frontier_detector_3d frontier_detector.launch.py,</code> alongside <code>octomap_server</code> with <code>publish_free_space:=true</code> .
Goal consumption	UAV nav node subscribes to <code>/frontiers/goals</code> (PoseArray, ranked by gain). Pop the first pose, fly there, repeat.
Phase 3 RL state	Import <code>FrontierDetector</code> directly in the Gymnasium env. No ROS round-trip during training.
Phase 4 AVCE	Extend <code>SparseVoxelGrid</code> with observation-angle metadata. Algorithm shape unchanged.
Phase 5 swarm	Run one detector per fused global map; UAVs auction over the ranked clusters.

11. Future Extensions

All of the following slot cleanly into the existing layer split — no architectural rework needed:

- **OctoMap binary message adapter.** Replace the dual-PointCloud2 source with a direct `octomap_msgs/Octomap` subscriber. One new file in infrastructure; zero domain changes.
- **Sphere-based information gain.** Replace the unknown-neighbour count with an integral over unknown volume in a sphere of radius R around the cluster centroid. More expensive but more accurate; pure-domain change.
- **Frontier cost function.** Combine info gain, distance from current pose (read TF), and altitude preference into a single utility. Pure-domain change.
- **Per-voxel observation-angle metadata.** Foundation for AVCE viewpoint diversity reward. Domain-layer extension.
- **Multi-resolution grids.** Coarse for exploration, fine for areas of interest. Adapter pattern in the domain layer.

12. Quick Start

From a clean ROS 2 jazzy install on Ubuntu 24.04:

```
# 1. Install dependencies
sudo apt install ros-jazzy-sensor-msgs-py ros-jazzy-rviz2 \
python3-numpy python3-pytest

# 2. Drop the package into a colcon workspace
cd ~/ros2_ws/src
```

```
git clone <your-repo-url> frontier_detector_3d

# 3. Build
cd ~/ros2_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select frontier_detector_3d
source install/setup.bash

# 4. Run the unit tests (algorithm verification)
cd src/frontier_detector_3d && python3 -m pytest test/ -v

# 5. Run the self-contained demo (no drone needed)
ros2 launch frontier_detector_3d demo.launch.py

# 6. Inspect the output
ros2 topic echo /frontiers/goals --once
ros2 topic hz /frontiers/markers
```

13. Conclusion

frontier_detector_3d is a small, focused, well-tested ROS 2 package that does exactly one thing well: detect 3D exploration frontiers in volumetric maps. Three things make it more than a tutorial implementation:

(1) Clean architecture is real, not decorative. The pure-domain algorithm has zero ROS imports and runs 26 unit tests in 0.12 seconds. The ROS layer is a thin adapter. The two are connected via abstract ports. Swapping ROS for any other transport is a one-file change.

(2) The demo runs out of the box. One launch file brings up a synthetic scene, the detector, RViz, and a static TF publisher. No drone, no SLAM, no octomap_server required. This makes the package portfolio-presentable on its own — the thirty-second elevator video is a single command away.

(3) It scales up cleanly into the AVCE project. The same code runs unchanged in Phase 2 (with a real OctoMap), Phase 3 (in a Gymnasium env during RL training), and Phase 5 (on a fused global map for the multi-UAV swarm). The Phase 4 AVCE extensions are well-localized to the domain layer, exactly where they should be.

Bottom line: a useful component on its own, a well-engineered foundation for the larger work to come. Built once, reused at every phase.

References

- [1] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *Proc. IEEE Int. Symp. Computational Intelligence in Robotics and Automation (CIRA)*, 1997, pp. 146–151.

- [2] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, “OctoMap: An efficient probabilistic 3D mapping framework based on octrees,” *Autonomous Robots*, vol. 34, no. 3, pp. 189–206, 2013.
- [3] B. Zhou, Y. Zhang, X. Chen, and S. Shen, “FUEL: Fast UAV exploration using incremental frontier structure and hierarchical planning,” *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 779–786, 2021.
- [4] A. Bircher, M. Kamel, K. Alexis, H. Oleynikova, and R. Siegwart, “Receding horizon ‘next-best-view’ planner for 3D exploration,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2016, pp. 1462–1468.
- [5] B. Lindqvist, A. Patel, K. Löfgren, and G. Nikolakopoulos, “A tree-based next-best-trajectory method for 3D UAV exploration,” arXiv preprint arXiv:2407.04386, 2024.
- [6] M. [Author Name], “Autonomous UAV swarm with articulated sensor platform for adaptive 3D mapping in unknown environments,” Technical project proposal, 2026.